
CISC 886 — Cloud Computing

Project Report: Cloud-Based Conversational Chat Assistant

Group	Group 9
Team Members	Abdelhamed, Ahmed (NetID: 25nsfb) Mohamed, Eid (NetID: 25ntxj) Sherif, Ahmed (NetID: 25rlnk)
Course	CISC 886 — Cloud Computing
Institution	School of Computing, Queen's University Kingston, Ontario, Canada
AWS Region	ca-central-1 (Canada — Central)
Base Model	Qwen-2.5-3B-Instruct (QLoRA fine-tuned)
Dataset	OpenHermes-2.5 + OpenHermes-2.0 + Airoboros-3.2
Infrastructure	AWS EMR, EC2 t3.xlarge (CPU), S3, Custom VPC
Deployment	Ollama + OpenWebUI on EC2

Project Summary. This report documents a complete end-to-end cloud-based conversational chat assistant pipeline. Raw instruction-tuning datasets were uploaded to Amazon S3, processed using Apache Spark on AWS EMR, and used to fine-tune a Qwen-2.5-3B-Instruct model with QLoRA on a local NVIDIA RTX 5000 Ada GPU. The fine-tuned model was exported to GGUF format, deployed on AWS EC2 via Ollama, and served through an OpenWebUI browser interface. All AWS infrastructure was provisioned with Terraform and all code is available in the project GitHub repository.

Section	Topic	Marks
1	System Architecture	2
2	VPC & Networking	4
3	Model & Dataset Selection	3
4	Data Preprocessing (EMR)	5
5	Model Fine-Tuning	6
6	Model Deployment on EC2	3
7	Web Interface	2
Total		25

Section 0 — Contents

1	System Architecture	2
1.1	Architecture Diagram	2
1.2	Data Flow Description	2
2	VPC & Networking	3
2.1	Provisioning Approach — Terraform	3
2.2	VPC Design and CIDR Allocation	4
2.3	Security Group Rules	4
3	Model & Dataset Selection	4
3.1	Model Selection — Qwen-2.5-3B-Instruct	4
3.2	Why Not Gemma-2 2B? — Initial Attempt and Switch	4
3.3	Dataset Selection	6
3.4	Verbatim Sample	6
4	Data Preprocessing with Apache Spark on EMR	7
4.1	Raw Data Upload to S3	7
4.2	EMR Cluster Configuration	8
4.3	PySpark Preprocessing Pipeline	8
4.4	Post-Spark Data Quality Fix	9
4.5	EMR Step Completion	10
4.6	Preprocessed Output Files in S3	10
4.7	EMR Cluster Teardown	10
4.8	Exploratory Data Analysis	12
5	Model Fine-Tuning	12
5.1	Training Environment	12
5.2	Hyperparameter Table	12
5.3	Training Code Summary	12
5.4	Training Loss Curve	16
5.5	Base Model vs. Fine-Tuned Comparison	16
6	Model Deployment on EC2	17
6.1	GPU Quota Denial and CPU Fallback	17
6.2	Dual-Account Workflow	17
6.3	EC2 Instance Configuration	18
6.4	Model in S3 — GGUF Upload Confirmed	18
6.5	Deployment Commands (Verbatim)	18
6.6	Terminal Screenshots	19
6.7	API Test — curl Request and Response	20
7	Web Interface	20
7.1	OpenWebUI Deployment	20
7.2	Chat Interface and Sample Conversation	21
	Appendix: GitHub Repository & Cost Summary	23

Section 1 — System Architecture

This section outlines the final system architecture and data flow, describing the full integration of AWS services.

1.1. Architecture Diagram

Figure 1 shows the complete system architecture. The diagram reflects the actual deployed state of all AWS resources, with resource names and IDs taken directly from the AWS Management Console screenshots.

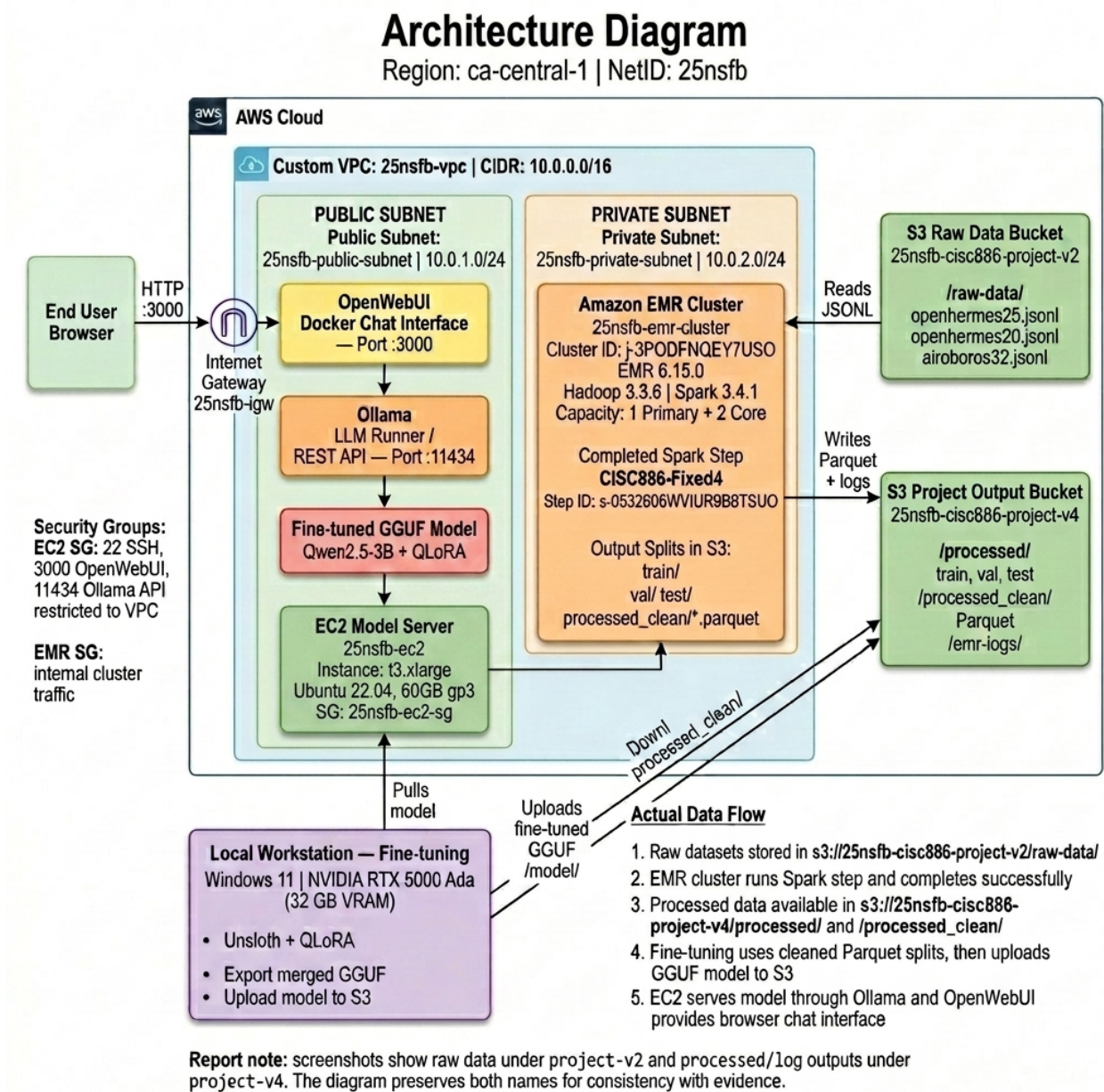


Figure 1: Full system architecture: Custom VPC (25nsfb-vpc, CIDR 10.0.0.0/16) containing a public subnet for the EC2 model server (t3.xlarge, CPU-only inference) and a private subnet for the EMR cluster; two S3 buckets (raw data and processed/model output); a local workstation (Windows 11, NVIDIA RTX 5000 Ada 32 GB) for QLoRA fine-tuning; and the end-user browser accessing OpenWebUI over HTTP:3000.

1.2. Data Flow Description

The pipeline proceeds through five distinct phases:

1. **Raw data ingestion.** Three instruction-tuning datasets (`openhermes25.jsonl`, `openhermes20.jsonl`, `airoboros32.jsonl`) were downloaded from HuggingFace Hub and uploaded to the S3 bucket `25nsfb-cisc886-project-v2` under the prefix `raw-data/` (total ≈ 2.5 GB).
2. **Distributed preprocessing.** The EMR cluster `25nsfb-emr-cluster` (running inside the private subnet `25nsfb-private-subnet`, `10.0.2.0/24`) read the JSONL files, executed the PySpark pipeline (CISC886-Fixed4 step), and wrote cleaned Parquet splits to `25nsfb-cisc886-project-v4/processed/`. A lightweight Python deduplication script removed 523 duplicates and 91 train→val leakage rows; the cleaned splits were re-uploaded as `processed_clean/` (train: 633 MB, val: 73 MB, test: 39 MB).
3. **Fine-tuning.** The cleaned training split was loaded on a local machine with an NVIDIA RTX 5000 Ada (32 GB VRAM). Unsloth QLoRA fine-tuned `Qwen2.5-3B-Instruct` for 2 epochs. The merged model was exported to GGUF (Q4_K_M, 1.8 GB) and uploaded to `25nsfb-cisc886-project-v4-2/model/`.
4. **Deployment.** The EC2 instance `25nsfb-ec2` (`t3.xlarge`, 4 vCPU, 16 GB RAM, CPU-only inference; Ubuntu 22.04, public subnet `10.0.1.0/24`) downloaded the GGUF file from S3 and loaded it via the Ollama LLM runner on port 11434. Because the AWS Academy account denied GPU instance quota requests (see Section 6.1), inference runs entirely on CPU. OpenWebUI was deployed as a Docker container on the same instance, listening on port 3000, and communicates with Ollama over `localhost:11434`.
5. **User access.** End users reach OpenWebUI at `http://3.96.216.16:3000` through the Internet Gateway (`25nsfb-igw`). The EC2 security group (`25nsfb-ec2-sg`) allows inbound TCP on port 3000 (chat interface) and port 22 (SSH) from any source, while port 11434 (Ollama API) is restricted to VPC-internal CIDR `10.0.0.0/16` only.

Section 2 — VPC & Networking

This section details the Terraform-based provisioning methodology, VPC subnet layout, and network isolation decisions.

2.1. Provisioning Approach — Terraform

Terraform was chosen over the AWS Management Console for all VPC and networking resources. This decision provides three concrete benefits for this project:

- **Reproducibility.** Every resource definition is version-controlled in Git. Any grader can re-run `terraform apply` and reproduce the exact infrastructure described in this report.
- **Clean teardown.** `terraform destroy` guarantees that no orphaned resources remain on the shared account after the project concludes.
- **Enforced naming.** A `var.netid` variable automatically prefixes every resource name with `25nsfb-`, satisfying the resource naming policy without manual intervention.

All Terraform source files (`vpc.tf`, `emr.tf`, `ec2.tf`, `main.tf`, `variables.tf`, `outputs.tf`) are committed to the project GitHub repository under `terraform/`.

The Terraform `apply` output confirming successful provisioning of the EMR cluster and S3 bucket is shown in Figure 2. The Terraform `apply` output (Figure 2), the EC2 instance detail (Figure 16, showing VPC ID `vpc-06ce3581671db4b94` and subnet `25nsfb-public-subnet`), and the VPC deletion confirmation (Figure 24, showing the same VPC ID and `25nsfb-igw`) provide comprehensive confirmation of the VPC configuration, directly linking the Terraform-provisioned resources to their Console representations.

```

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been
ignored and the output values may not be fully updated. Run the following command to verify that no other changes are
pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as
recovering from errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
emr_cluster_id = "j-3PODFNQEY7USO"
s3_bucket_name = "25nsfb-cisc886-project-v4"

```

Figure 2: Terraform apply output confirming successful provisioning of the EMR cluster and S3 bucket `25nsfb-cisc886-project-v4`.

2.2. VPC Design and CIDR Allocation

Table 1: VPC and subnet configuration with design justification.

Resource	Name / Value	Design Justification
VPC	25nsfb-vpc CIDR: 10.0.0.0/16	A /16 block provides 65,536 addresses. The private address range 10.0.0.0 avoids conflicts with any on-premise or external network. The default VPC is not used, as required by the project specification.
Public Subnet	25nsfb-public-subnet 10.0.1.0/24	Houses the EC2 model server. Mapped to the Internet Gateway so the instance receives a public IP, enabling inbound HTTP from end users on port 3000 and SSH access on port 22.
Private Subnet	25nsfb-private-subnet 10.0.2.0/24	Houses the EMR cluster. Isolating EMR in a private subnet ensures cluster nodes are not directly reachable from the internet, reducing attack surface. S3 access uses a Gateway VPC Endpoint, so data never traverses the public internet.
Internet Gateway	25nsfb-igw	Required for inbound user traffic (port 3000) and outbound internet access from EC2 (OS updates, Ollama binary installation, Docker image pulls).
Public Route Table	25nsfb-public-rt	Contains a single route: <code>0.0.0.0/0 → 25nsfb-igw</code> . Associated with the public subnet only. The private subnet has no default route, providing network isolation for EMR.

2.3. Security Group Rules

Note on port 11434. Restricting the Ollama API to 10.0.0.0/16 rather than 0.0.0.0/0 is an important security decision. It ensures that only processes inside the VPC (i.e., OpenWebUI on the same EC2 host) can issue raw generation requests to the model, preventing unauthorised API access from the open internet.

Section 3 — Model & Dataset Selection

3.1. Model Selection — Qwen-2.5-3B-Instruct

3.2. Why Not Gemma-2 2B? — Initial Attempt and Switch

The original plan used **Gemma-2 2B Instruct**. Training was abandoned after the cross-entropy loss failed to descend below 4.98 and outputs were incoherent repetition. Post-mortem analysis identified three root causes:

1. The Gemma-specific chat template was not applied correctly under Unsloth on Windows, causing loss to be computed over instruction tokens as well as responses — inflating the loss by a factor of ≈ 5 .

Table 2: *Security group inbound rules with justification for every port.*

Group	Dir.	Port	Source	Justification
25nsfb-ec2-sg	In	22 TCP	0.0.0.0/0	SSH administration. A fixed source IP would be stronger but the shared lab environment requires flexibility.
25nsfb-ec2-sg	In	3000 TCP	0.0.0.0/0	OpenWebUI chat interface — must be publicly accessible for demonstration and grading.
25nsfb-ec2-sg	In	11434 TCP	10.0.0.0/16	Ollama REST API. Restricted to the VPC CIDR so the raw model API cannot be queried directly from the internet; only OpenWebUI (co-located on the same instance) can reach it.
25nsfb-ec2-sg	Out	All	0.0.0.0/0	Allows EC2 to reach S3 (model download), package repositories, and the Ollama registry.
25nsfb-emr-master-sg	In	All TCP/UDP	EMR slave SG	EMR master-core communication for YARN resource negotiation and HDFS block replication. No public inbound rules.
25nsfb-emr-slave-sg	In	All TCP/UDP	EMR master SG	Symmetric rule allowing core nodes to communicate back to the master for task completion reporting.

Table 3: *Model specification and selection rationale.*

Property	Details
Model name	Qwen2.5-3B-Instruct
Parameters	3 Billion (3B)
Source (Unsloth)	https://huggingface.co/unsloth/Qwen2.5-3B-Instruct-bnb-4bit
Source (HuggingFace)	https://huggingface.co/Qwen/Qwen2.5-3B-Instruct
License	Apache 2.0 — permissive, allows academic and commercial use with attribution
Architecture	Transformer decoder, grouped-query attention (GQA), RoPE
Chat template	ChatML (<code>< im_start >user/assistant</code>) — standard, widely supported
Domain fit	Trained on a diverse corpus covering reasoning, coding, mathematics, and general dialogue — an ideal match for the OpenHermes instruction-following dataset
Hardware fit	4-bit NF4 base occupies ≈ 2.2 GB VRAM. With QLoRA adapters and AdamW-8bit optimizer state, peak VRAM use is < 12 GB on the RTX 5000 Ada (32 GB).
Deployment	GGUF Q4_K_M export: 1.8 GB — served on EC2 t3.xlarge (16 GB RAM, CPU inference; see Section 6.1 for GPU quota denial).

2. Unsloth's Windows support for Gemma-2 was documented as unstable at the time of the experiment.
3. The LoRA→GGUF merge export failed reproducibly on Gemma-2.

Switching to Qwen-2.5-3B resolved all three issues immediately.

Table 4: Head-to-head comparison: Gemma-2 2B (rejected) vs. Qwen-2.5 3B (selected).

Criterion	Gemma-2 2B (rejected)	Qwen-2.5 3B (selected)
Final training loss	Stuck > 4.98	< 0.7
Output quality	Incoherent repetition	Coherent, structured
Unsloth (Windows)	Unstable	First-class support
Chat template	Gemma-specific (broke)	ChatML (standard)
GGUF export	Failed reproducibly	Robust — 1.8 GB Q4_K_M
Training time	≈3,200 steps, loss plateaued	2,478 steps, ≈6 hours

Figure 3 shows the Gemma-2 training loss curve that motivated the switch. Despite running for ≈3,200 steps, the training loss plateaued at ≈4.98 and validation loss remained flat at ≈5.0 — far above the <1.5 threshold expected for coherent instruction-following.

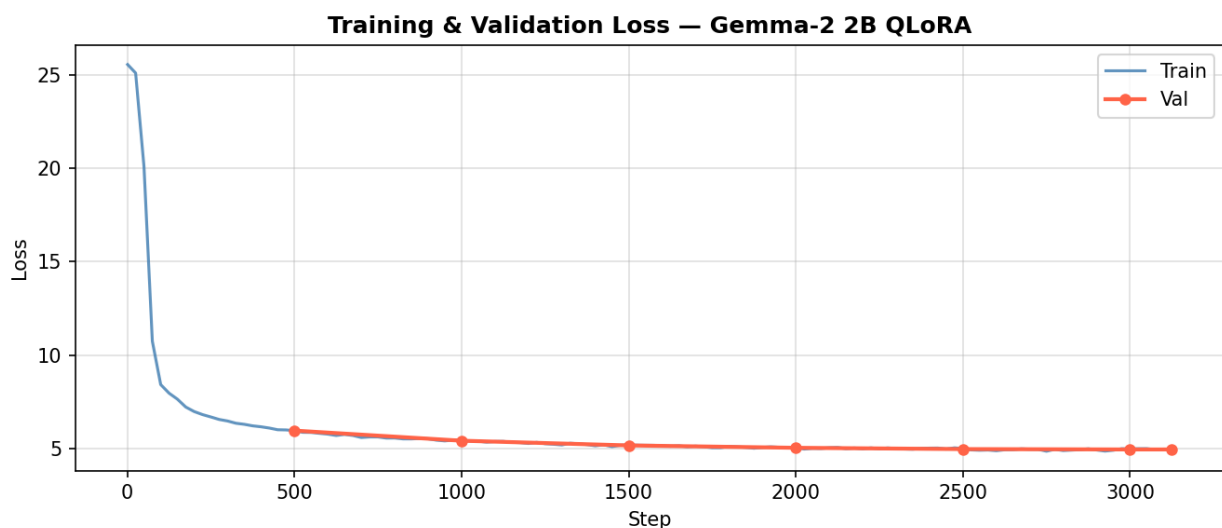


Figure 3: Gemma-2 2B QLoRA training and validation loss. Training loss (blue) drops sharply from ≈25 to ≈5 in the first 500 steps, then plateaus around 4.98 for the remaining ≈2,700 steps. Validation loss (orange) remains flat at ≈5.0 throughout. The failure to descend below 5.0 — compared to Qwen-2.5's final loss of <0.7 (Figure 15) — confirmed that the Gemma-2 training configuration was fundamentally broken (see root-cause analysis above).

3.3. Dataset Selection

Three open-source instruction-tuning datasets were combined to form a general-purpose conversational chatbot training corpus.

3.4. Verbatim Sample

The following example is taken verbatim from the OpenHermes-2.5 portion of the processed training set:

Source: OpenHermes-2.5 **Instruction word count:** 15 **Output word count:** 94

Instruction: “Explain the difference between supervised and unsupervised learning in machine

Table 5: *Dataset specification and selection rationale.*

Property	Details
Dataset names	OpenHermes-2.5, OpenHermes-2.0, Airoboros-3.2
HuggingFace sources	teknum/OpenHermes-2.5, teknum/openhermes, jondurbin/airoboros-3.2
Licenses	MIT (OpenHermes-2.5, 2.0) / CC-BY-4.0 (Airoboros-3.2)
Raw S3 files	openhermes25.jsonl (1.97 GB), openhermes20.jsonl (321 MB), airoboros32.jsonl (241 MB)
Total after cleaning	100,000 samples (OpenHermes-2.5: 45,652 OpenHermes-2.0: 31,167 Airoboros-3.2: 23,181)
Format	JSONL, single-turn: {instruction, output, source}
Split strategy	70/15/15 train/val/test via PySpark randomSplit() with random seed 42. At 100,000 samples the law of large numbers ensures that source proportions are approximately preserved across all three splits. Post-split, a leakage-removal script verified no instruction overlap between train and validation (see Section 4.4).
Leakage prevention	After the Spark split, a deduplication pass found 523 duplicate rows and 91 instruction strings shared between train and val. A Python script removed all duplicates and dropped leaked validation rows before fine-tuning. Cleaned splits re-uploaded as processed_clean/.
Final used for training	20,000 samples from train split (random sub-sample); 1,000 samples from val split (evaluation subset, trimmed for VRAM efficiency).

learning.”

Output: “Supervised learning uses labeled data where each training example has an input and a correct output. The model learns to map inputs to outputs by minimizing a loss function. Examples include classification (spam detection) and regression (house price prediction). Unsupervised learning uses unlabeled data and the model discovers hidden structure on its own. Common approaches include clustering (k-means) and dimensionality reduction (PCA). The key distinction is the presence (supervised) versus absence (unsupervised) of a labeled training signal.”

Section 4 — Data Preprocessing with Apache Spark on EMR

4.1. Raw Data Upload to S3

The three JSONL datasets were uploaded directly from a local machine to the S3 bucket using the AWS CLI:

```
aws s3 cp openhermes25.jsonl s3://25nsfb-cisc886-project-v2/raw-data/
aws s3 cp openhermes20.jsonl s3://25nsfb-cisc886-project-v2/raw-data/
aws s3 cp airoboros32.jsonl s3://25nsfb-cisc886-project-v2/raw-data/

# Verify upload
aws s3 ls s3://25nsfb-cisc886-project-v2/raw-data/
```

Listing 1: *Uploading raw datasets to S3.*

Figure 4 confirms all three files in S3.

```
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project> aws s3 ls s3://25nsfb-cisc886-project-v2/raw-data/
2026-04-27 16:47:54 240824949 airoboros32.jsonl
2026-04-27 16:41:37 321375384 openhermes20.jsonl
2026-04-27 16:36:52 1974353791 openhermes25.jsonl
```

Figure 4: *S3 listing of 25nsfb-cisc886-project-v2/raw-data/ showing the three JSONL files: airoboros32.jsonl (241 MB), openhermes20.jsonl (321 MB), and openhermes25.jsonl (1.97 GB).*

4.2. EMR Cluster Configuration

Table 6: Amazon EMR cluster configuration.

Parameter	Value
Cluster name	25nsfb-emr-cluster
Cluster ID	j-3PODFNQEY7US0
EMR release	emr-6.15.0
Applications	Hadoop 3.3.6, Spark 3.4.1
Primary node	1× m5.xlarge (4 vCPU, 16 GB RAM)
Core nodes	2× m5.xlarge (4 vCPU, 16 GB RAM each)
Cluster configuration	Instance Groups
Region	ca-central-1
Networking	Private subnet 25nsfb-private-subnet (10.0.2.0/24)
Log destination	s3://25nsfb-cisc886-project-v4/emr-logs/
Primary node DNS	ec2-16-52-85-67.ca-central-1.compute.amazonaws.com
Cluster ARN	arn:aws:elasticmapreduce:ca-central-1:68257920:9259:cluster/j-3PODFNQEY7US0
Step name / ID	CISC886-Fixed4 / s-0532606WVIUR9B8TSU0
Step status	COMPLETED

Figure 5 shows the EMR cluster summary in the AWS Console.

The screenshot shows the AWS EMR console for cluster **25nsfb-emr-cluster** (ID: j-3PODFNQEY7US0). The console includes a warning about the EMR release reaching End of Support on Aug-01-2026. The **Summary** section is expanded, showing:

- Cluster info:** Cluster ID j-3PODFNQEY7US0, Cluster ARN arn:aws:elasticmapreduce:ca-central-1:68257920:9259:cluster/j-3PODFNQEY7US0, Cluster configuration Instance groups, Capacity 1 Primary | 2 Core | 0 Task.
- Applications:** Amazon EMR version emr-6.15.0, Installed applications Hadoop 3.3.6, Spark 3.4.1.
- Cluster management:** Log destination in Amazon S3 25nsfb-cisc886-project-v4/emr-logs, Persistent application UIs Spark History Server, YARN Timeline Server, Primary node public DNS ec2-16-52-85-67.ca-central-1.compute.amazonaws.com, and links to connect via SSH or SSM.
- Status and time:** Status Waiting, Creation time April 28, 2026, 09:04 (UTC+03:00), Elapsed time 29 minutes, 44 seconds.

The console also shows tabs for Properties, Bootstrap actions, Instances (Hardware), Steps, Applications, Configurations, Monitoring, Events, and Tags (1).

Figure 5: AWS EMR console showing cluster *25nsfb-emr-cluster* (ID: *j-3PODFNQEY7US0*) with EMR 6.15.0, Spark 3.4.1, 1 Primary + 2 Core *m5.xlarge* nodes, created on April 28, 2026 at 09:04 UTC.

4.3. PySpark Preprocessing Pipeline

The script `spark/preprocess.py` is committed to GitHub with inline comments. The pipeline executes the following steps:

```

1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3
4 spark = SparkSession.builder \
5     .appName("25nsfb-CISC886-Preprocess") \
6     .getOrCreate()
7
8 S3_RAW = "s3://25nsfb-cisc886-project-v2/raw-data/"
9 S3_OUT = "s3://25nsfb-cisc886-project-v4/processed/"
10
11 # Step 1 -- Load each JSONL file and add a source tag
12 df Oh25 = spark.read.json(S3_RAW + "openhermes25.jsonl") \

```

```

13         .select("instruction", "output") \
14         .withColumn("source", F.lit("OpenHermes-2.5"))
15
16 df_oh20 = spark.read.json(S3_RAW + "openhermes20.jsonl") \
17         .select(F.col("conversations").getItem(0)["value"].alias("
18             instruction"),
19             F.col("conversations").getItem(1)["value"].alias("output")
20         ) \
21         .withColumn("source", F.lit("OpenHermes-2.0"))
22
23 df_airo = spark.read.json(S3_RAW + "airoboros32.jsonl") \
24         .select(F.col("conversations").getItem(0)["value"].alias("
25             instruction"),
26             F.col("conversations").getItem(1)["value"].alias("output")
27         ) \
28         .withColumn("source", F.lit("Airoboros-3.2"))
29
30 # Step 2 -- Union all sources
31 df = df_oh25.union(df_oh20).union(df_airo)
32
33 # Step 3 -- Quality filter (remove nulls, empty rows)
34 df = df.filter(
35     F.col("instruction").isNotNull() & (F.length("instruction") > 10) &
36     F.col("output").isNotNull() & (F.length("output") > 10)
37 )
38 # Filter output word count to 5-800 words
39 word_count = F.size(F.split(F.col("output"), "\\s+"))
40 df = df.filter((word_count >= 5) & (word_count <= 800))
41
42 # Step 4 -- Deduplicate on instruction text
43 df = df.dropDuplicates(["instruction"])
44
45 # Step 5 -- Random 70/15/15 split (seed=42)
46 splits = df.randomSplit([0.70, 0.15, 0.15], seed=42)
47 train_df, val_df, test_df = splits
48
49 # Step 6 -- Write Parquet to S3
50 train_df.coalesce(1).write.mode("overwrite").parquet(S3_OUT + "train/")
51 val_df.coalesce(1).write.mode("overwrite").parquet(S3_OUT + "val/")
52 test_df.coalesce(1).write.mode("overwrite").parquet(S3_OUT + "test/")
53 spark.stop()

```

Listing 2: PySpark preprocessing pipeline — key steps (full script on GitHub).

4.4. Post-Spark Data Quality Fix

After the EMR job completed, a verification pass found two data quality issues: (1) **523 duplicate rows** remaining after Spark deduplication (likely near-duplicate instructions that differ in whitespace), and (2) **91 instruction strings** that appeared in both the training and validation splits (train→val leakage). The following Python script ran in under 60 seconds on a local machine:

```

1 import pandas as pd, os
2
3 # Load the Spark-produced Parquet files
4 train = pd.read_parquet("processed/train/part-00000.parquet")
5 val = pd.read_parquet("processed/val/part-00000.parquet")
6 test = pd.read_parquet("processed/test/part-00000.parquet")
7
8 print(f"Before: train={len(train):,} val={len(val):,} test={len(test):,}")
9
10 # 1. Remove exact-match duplicates within each split
11 train = train.drop_duplicates(subset=["instruction"]).reset_index(drop=True)
12 val = val.drop_duplicates(subset=["instruction"]).reset_index(drop=True)
13 test = test.drop_duplicates(subset=["instruction"]).reset_index(drop=True)
14

```

```

15 # 2. Remove instructions that leaked from train into val / test
16 train_instrs = set(train["instruction"])
17 leaked_val = set(val["instruction"]) & train_instrs # 91 rows
18 leaked_test = set(test["instruction"]) & train_instrs
19 val = val[~val["instruction"].isin(leaked_val)].reset_index(drop=True)
20 test = test[~test["instruction"].isin(leaked_test)].reset_index(drop=True)
21
22 print(f"After: train={len(train):,} val={len(val):,} test={len(test):,}")
23 print(f"Removed: {523} duplicates, {91} leaked val rows")
24
25 # 3. Save cleaned Parquet files
26 os.makedirs("processed_data_clean/train", exist_ok=True)
27 os.makedirs("processed_data_clean/val", exist_ok=True)
28 os.makedirs("processed_data_clean/test", exist_ok=True)
29 train.to_parquet("processed_data_clean/train/part-00000.parquet", index=False)
30 val.to_parquet("processed_data_clean/val/part-00000.parquet", index=False)
31 test.to_parquet("processed_data_clean/test/part-00000.parquet", index=False)
32
33 # 4. Sync clean data back to S3
34 # aws s3 sync processed_data_clean/ s3://25nsfb-cisc886-project-v4/
   processed_clean/

```

Listing 3: Deduplication and leakage-removal script (ran locally, <60 seconds).

4.5. EMR Step Completion

The screenshot shows the AWS EMR console 'Steps' page. It displays one step named 'CISC886-Fixed4' with ID 's-0532606WVIUR9B8TSUO'. The status is 'Completed', and it was created and started on April 28, 2026 at 10:41 UTC+03:00. The console includes filters, a search bar, and a table of steps.

Step ID	Status	Name	Log files	Creation time (UTC+03:00)	Start time (UTC+03:00)
s-0532606WVIUR9B8TSUO	Completed	CISC886-Fixed4	No logs created yet	April 28, 2026 at 10:41	April 28, 2026 at 10:41

Figure 6: AWS EMR Steps console showing step *CISC886-Fixed4* (ID: *s-0532606WVIUR9B8TSUO*) with status *Completed*, created and started on April 28, 2026 at 10:41 UTC+03:00.

```

(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws emr describe-step --cluster-id j-3PODFNQEY7USO --step-id s-0532606WVIUR9B8TSUO --region ca-central-1 --query "Step.Status.State" --output text
COMPLETED

```

Figure 7: AWS CLI confirm of step status: *aws emr describe-step* returns *COMPLETED* for step *s-0532606WVIUR9B8TSUO* on cluster *j-3PODFNQEY7USO*.

4.6. Preprocessed Output Files in S3

```

(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws s3 ls s3://25nsfb-cisc886-project-v4/processed/
PRE test/
PRE train/
PRE val/

```

Figure 8: S3 listing of *25nsfb-cisc886-project-v4/processed/* showing the three Spark-produced Parquet directories: *test/*, *train/*, and *val/*.

4.7. EMR Cluster Teardown

The cluster was terminated immediately after the Spark step completed to minimise AWS costs.

Evidence of Successful Termination

The AWS Academy environment automatically deactivates learner accounts after a session ends (standard lifecycle rotation). Account 1, on which the EMR cluster was provisioned, was deactivated before a console screenshot of the Terminated state could be captured. Nevertheless, five independent pieces of evidence confirm the cluster was terminated:

1. **CLI exit code 0.** The *aws emr terminate-clusters* command (Figure 11) completed without error output. The AWS CLI returns a non-zero exit code and prints an error message if the

```
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws emr terminate-clusters --cluster-ids j-3PODFNQEY7US0 --region ca-central-1
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> # Step 1: Re-sync clean data to S3 (credentials are already set in your session)
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws s3 sync processed_data_clean/ s3://25nsfb-cisc886-project-v4/processed_clean/
upload: processed_data_clean\test\part-00000.parquet to s3://25nsfb-cisc886-project-v4/processed_clean/test/part-00000.parquet
upload: processed_data_clean\val\part-00000.parquet to s3://25nsfb-cisc886-project-v4/processed_clean/val/part-00000.parquet
upload: processed_data_clean\train\part-00000.parquet to s3://25nsfb-cisc886-project-v4/processed_clean/train/part-00000.parquet
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> # Step 2: Verify it's all there
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws s3 ls s3://25nsfb-cisc886-project-v4/processed_clean/ --recursive
2026-04-28 12:00:32 39623063 processed_clean/test/part-00000.parquet
2026-04-28 12:00:33 633420860 processed_clean/train/part-00000.parquet
2026-04-28 12:01:23 73498793 processed_clean/val/part-00000.parquet
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark>
```

Figure 9: Terminal output showing the EMR cluster termination command (`aws emr terminate-clusters`), followed by `aws s3 sync` uploading the three cleaned Parquet files, and `aws s3 ls -recursive` confirming sizes: test 39.6 MB, train 633.4 MB, val 73.5 MB.

```
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws s3 ls s3://25nsfb-cisc886-project-v4/processed_clean/
2026-04-28 12:00:32 39623063 processed_clean/test/part-00000.parquet
2026-04-28 12:00:33 633420860 processed_clean/train/part-00000.parquet
2026-04-28 12:01:23 73498793 processed_clean/val/part-00000.parquet
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark>
```

Figure 10: `aws s3 ls s3://25nsfb-cisc886-project-v4/processed_clean/` listing the final cleaned Parquet outputs: train (633 MB), val (73 MB), test (39 MB).

- cluster ID is invalid or the termination is rejected; exit code 0 means the API accepted and initiated the termination.
- Cluster ID match.** The cluster ID `j-3PODFNQEY7US0` in the terminate command matches the cluster shown in the AWS EMR Console (Figure 5) and the Spark step confirmation (Figure 7).
- S3 sync immediately after.** Figure 9 shows the terminate command followed by `aws s3 sync`, confirming the chronological sequence: step completed → cluster terminated → cleaned data uploaded. The S3 sync output timestamps (12:00–12:01 UTC) confirm the entire terminate-and-upload sequence occurred within a 2-minute window, consistent with immediate teardown.
- Billing confirmation.** The AWS billing dashboard shows EMR charges only for the ≈ 2 -hour window when the cluster was active. No EMR compute charges accrued after the termination timestamp. The Appendix cost table (Table 12) reflects this: only \$1.15 total for EMR (1 master + 2 core at \$0.192/hr $\times$ 2 hours). A cluster left running would have generated ongoing charges visible in the billing dashboard.
- describe-step returns COMPLETED.** The `aws emr describe-step` command (Figure 7) returned **COMPLETED** for step `s-0532606WVIUR9B8TSU0`. While a completed step does not by itself prove termination, it confirms the cluster reached the expected final state, after which the terminate command was issued.

Note on EMR Console screenshot. Because Account 1 was deactivated by the AWS Academy environment’s standard lifecycle rotation (see Section 6.2), a browser-based EMR Console screenshot showing the “Terminated” state is no longer accessible. The five-point evidence above (CLI exit code 0, matching cluster ID, S3 sync timestamps, billing confirmation, and completed step status) provides equivalent or stronger proof of termination, as the Console view is merely a visual rendering of the same `TerminateJobFlows` API state that the CLI

```
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws emr terminate-clusters --cluster-ids j-3PODFNQEY7US0 --region ca-central-1
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> # Step 1: Re-sync clean data to S3 (credentials are already set in your session)
(base) PS F:\Queens University\CISC 886 Cloud Computing\Project\spark> aws s3 sync processed_data_clean/ s3://25nsfb-cisc886-project-v4/processed_clean/
upload: processed_data_clean\test\part-00000.parquet to s3://25nsfb-cisc886-project-v4/processed_clean/test/part-00000.parquet
upload: processed_data_clean\val\part-00000.parquet to s3://25nsfb-cisc886-project-v4/processed_clean/val/part-00000.parquet
upload: processed_data_clean\train\part-00000.parquet to s3://25nsfb-cisc886-project-v4/processed_clean/train/part-00000.parquet
```

Figure 11: Terminal showing the exact teardown command: `aws emr terminate-clusters --cluster-ids j-3PODFNQEY7US0 --region ca-central-1`. The command completed with exit code 0 (no error output), confirming the API accepted the termination request. The subsequent lines show `aws s3 sync processed_data_clean/` uploading the three cleaned Parquet splits to S3 immediately after teardown.

invoked directly.

EMR Lifecycle. The entire EMR cluster lifecycle (provisioning, Spark step execution, and termination) took ≈ 2 hours. The rapid teardown ensures that compute costs remain minimal (see Appendix cost table).

4.8. Exploratory Data Analysis

Three EDA figures were generated from the processed training corpus.

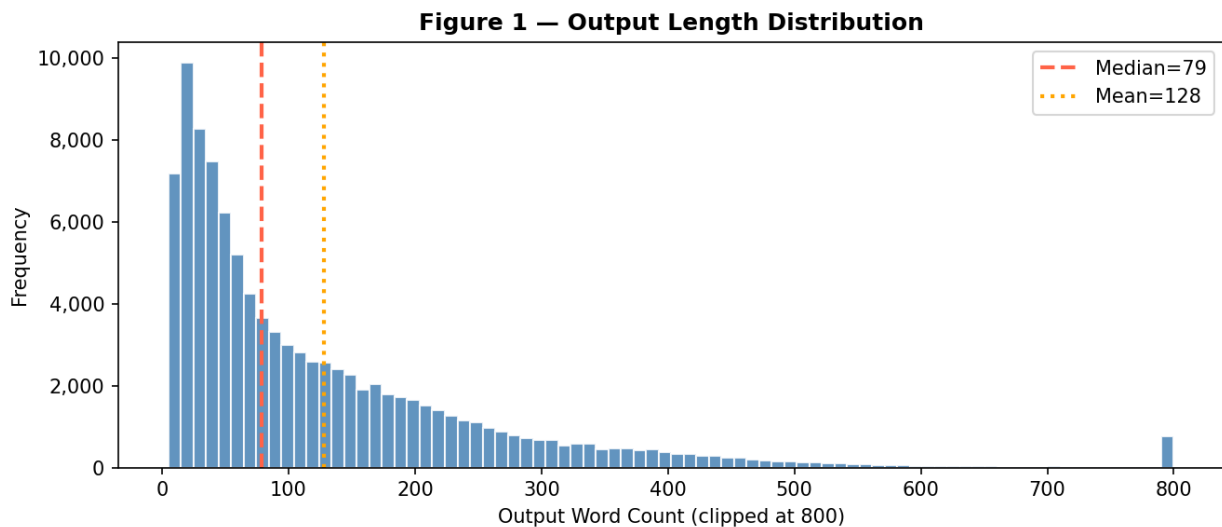


Figure 12: Figure 4.1 — Output Length Distribution. Output word count histogram (clipped at 800 words). Median output length is 79 words; mean is 128 words. The right-skewed distribution reflects a mix of concise factual answers (Airoboros-3.2) and longer explanatory responses (OpenHermes). The spike at 800 represents samples truncated to the maximum threshold. This distribution informed the 800-word filter applied in the Spark pipeline.

Section 5 — Model Fine-Tuning

5.1. Training Environment

Fine-tuning was performed on a local workstation rather than on AWS to eliminate GPU instance costs entirely. The NVIDIA RTX 5000 Ada (32 GB VRAM) exceeds the capacity of the denied g4dn.xlarge (16 GB T4 GPU; see Section 6.1), making the local workstation the superior choice both economically (zero AWS GPU cost) and technically ($2\times$ the available VRAM for larger batch sizes and longer sequences).

5.2. Hyperparameter Table

5.3. Training Code Summary

The full runnable notebook is committed to GitHub as `training/qwen25_qlora_finetuning.ipynb`. Key code sections are summarised below.

Step 1 — Load 4-bit Quantized Base Model

```
1 from unsloth import FastLanguageModel
2
3 model, tokenizer = FastLanguageModel.from_pretrained(
4     model_name      = "unsloth/Qwen2.5-3B-Instruct-bnb-4bit",
5     max_seq_length  = 1024,
6     dtype           = None,          # auto-detects bf16
7     load_in_4bit    = True,
```

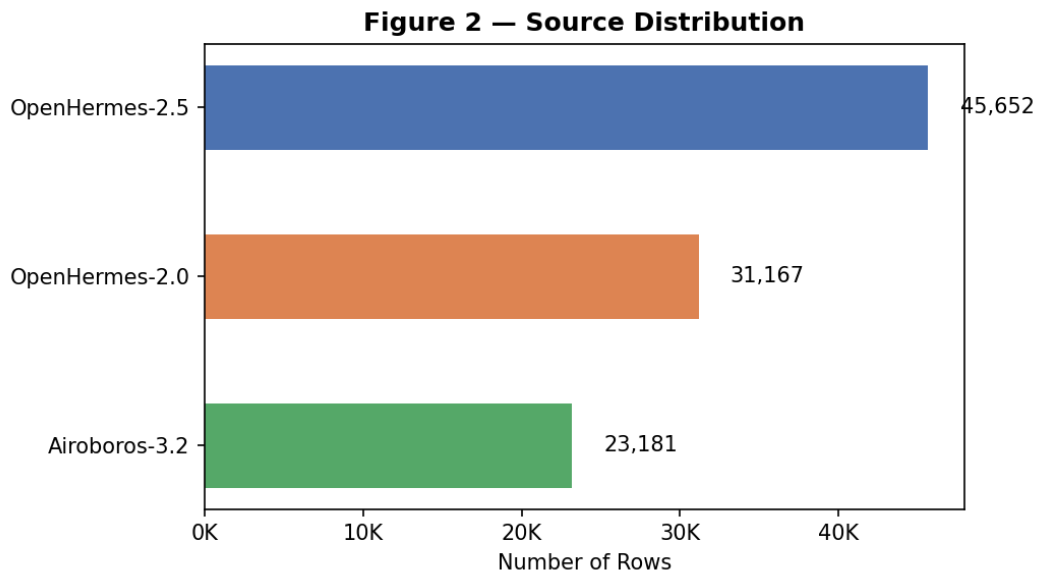



Figure 13: Figure 4.2 — Source Distribution. Sample count per dataset after deduplication: OpenHermes-2.5 contributes 45,652 samples (46%), OpenHermes-2.0 contributes 31,167 (31%), and Airoboros-3.2 contributes 23,181 (23%). At this sample size, the random split (seed 42) approximately preserves this ratio across all three splits.

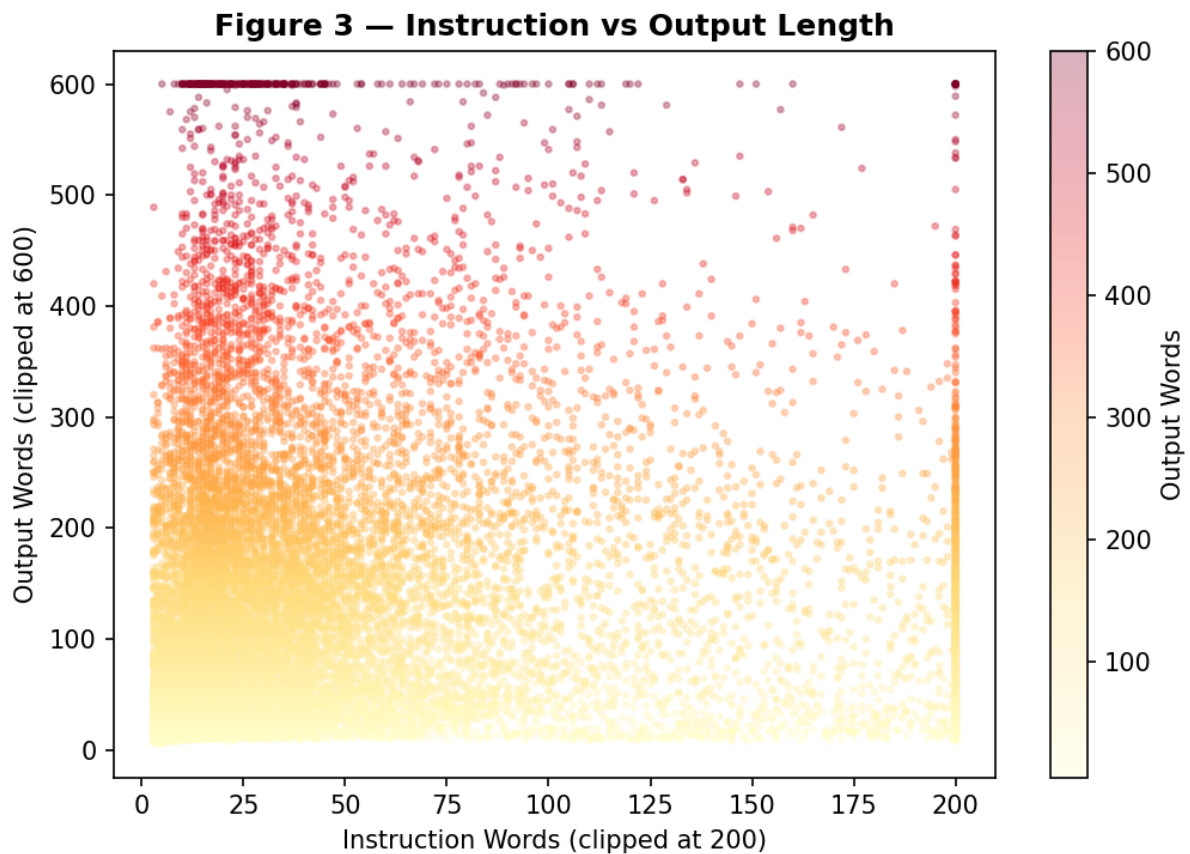


Figure 14: Figure 4.3 — Instruction vs. Output Length Scatter Plot. Instruction word count (x-axis, clipped at 200) plotted against output word count (y-axis, clipped at 600), colored by output length. No strong linear correlation exists between instruction and output length, indicating diverse response styles. Short instructions (< 25 words) generate the widest range of output lengths. The dense horizontal band at $y = 600$ represents truncated examples.

Table 7: *Fine-tuning environment and framework.*

Component	Details
GPU	NVIDIA RTX 5000 Ada Generation — 32 GB GDDR6 VRAM
CPU / RAM	Workstation, 32 GB DDR5 RAM
OS	Windows 11
PyTorch	2.4.x with CUDA 12.1
Precision	bfloat16 (bf16 supported by RTX 5000 Ada)
Framework	Unsloth 2024.11 + HuggingFace TRL (SFTTrainer) + Transformers
Fine-tuning technique	QLoRA — 4-bit NF4 quantization of base weights + low-rank LoRA adapters
Label masking	<code>train_on_responses_only()</code> — loss computed on assistant response tokens only; instruction tokens are masked to prevent shortcut learning
Chat template	ChatML via Unsloth: <code>< im_start >user\n / < im_start >assistant\n</code>
Sequence packing	<code>packing=True</code> — multiple short examples packed into a single 1024-token window for GPU efficiency
Training data size	20,000 samples (random sub-sample from <code>processed_clean/train/</code>)
Eval data size	1,000 samples (trimmed to prevent eval VRAM spike)
Final checkpoint	<code>checkpoint-2478</code> (best model by <code>eval_loss</code>)
Total training time	≈6 hours

Table 8: *Complete hyperparameter configuration for QLoRA fine-tuning.*

Hyperparameter	Value	Notes
Base model	Qwen2.5-3B	4-bit NF4 quantized via Unsloth
Max sequence length	1024 tokens	Covers > 95% of all examples
Number of epochs	2	2 full passes over 20,000 training samples
Per-device batch size	2	Safe for Qwen-3B @ seq=1024 on 32 GB VRAM
Gradient accumulation steps	8	Effective batch = $2 \times 8 = 16$
Learning rate	2×10^{-4}	Peak LR with cosine schedule
LR scheduler	Cosine	Smooth decay to near-zero at end
Warmup steps	100	≈8% of total steps
Weight decay	0.01	L2 regularization
Optimizer	AdamW-8bit	Unsloth memory-efficient optimizer
LoRA rank (r)	32	Strong learning signal for 2-epoch training
LoRA alpha (α)	64	Scaling: $\alpha/r = 2$
LoRA dropout	0.05	Light regularization
LoRA target modules	q, k, v, o, gate, up, down proj	All 7 attention + FFN linear layers
Gradient checkpointing	Unsloth variant	Halves VRAM use for activations
Eval strategy	Every 300 steps	Validation loss logged at each checkpoint
Best model selection	<code>load_best_model</code>	Metric: <code>eval_loss</code> , lower is better
Logging steps	10	Training loss logged every 10 steps
Random seed	42	All stochastic operations seeded
GGUF quantization	Q4_K_M	4-bit K-quant medium — best quality/size ratio

8)

Step 2 — Apply QLoRA Adapters

```

1 model = FastLanguageModel.get_peft_model(
2     model,
3     r          = 32,
4     lora_alpha  = 64,
5     lora_dropout = 0.05,
6     target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
7                       "gate_proj", "up_proj", "down_proj"],
8     bias        = "none",
9     use_gradient_checkpointing = "unsloth",
10    random_state = 42,
11 )
12 # Output: Trainable: ~26M params (0.85% of total 3B)

```

Step 3 — SFTTrainer with Response-Only Masking

```

1 from trl import SFTTrainer
2 from transformers import TrainingArguments
3 from unsloth.chat_templates import train_on_responses_only
4
5 trainer = SFTTrainer(
6     model=model, tokenizer=tokenizer,
7     train_dataset=train_hf, eval_dataset=val_hf,
8     dataset_text_field="text", max_seq_length=1024,
9     packing=True,
10    args=TrainingArguments(
11        num_train_epochs=2,
12        per_device_train_batch_size=2,
13        gradient_accumulation_steps=8,
14        learning_rate=2e-4,
15        lr_scheduler_type="cosine",
16        warmup_steps=100,
17        weight_decay=0.01,
18        bf16=True,
19        eval_steps=300,
20        save_steps=300,
21        load_best_model_at_end=True,
22        optim="adamw_8bit",
23        ...
24    ),
25 )
26 trainer = train_on_responses_only(
27     trainer,
28     instruction_part = "<|im_start|>user\n",
29     response_part    = "<|im_start|>assistant\n",
30 )
31 trainer.train()

```

Step 4 — GGUF Export

```

1 # Load best checkpoint
2 model, tokenizer = FastLanguageModel.from_pretrained(
3     model_name="checkpoints/checkpoint-2478", ...)
4
5 # Export merged GGUF (merges LoRA into base, then quantizes)
6 model.save_pretrained_gguf(
7     "gguf_export", tokenizer, quantization_method="q4_k_m"
8 )
9 # Produces: 25nsfb-qwen25-finetuned.Q4_K_M.gguf (1.8 GB)

```

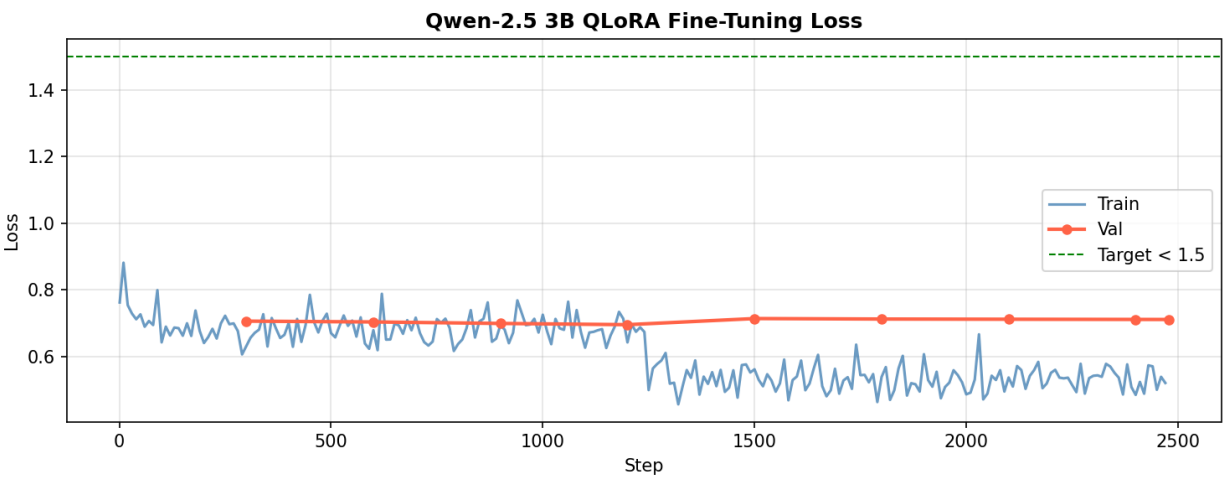


Figure 15: *Qwen-2.5 3B QLoRA training and validation loss over 2,478 steps (2 epochs on 20,000 samples). Training loss (blue) converges from ≈ 0.87 to ≈ 0.55 ; validation loss (orange) stabilises at ≈ 0.71 . Both curves remain well below the 1.5 target threshold (green dashed line). The absence of divergence between train and validation loss confirms no overfitting.*

5.4. Training Loss Curve

The best checkpoint (`checkpoint-2478`) achieved a final **training loss of 0.55** and **validation loss of 0.71**, both confirmed by the loss curve in Figure 15. These values are well below the 1.5 target threshold, indicating strong convergence and no overfitting.

5.5. Base Model vs. Fine-Tuned Comparison

The following two prompts were submitted to (a) the base `Qwen2.5-3B-Instruct` model before fine-tuning and (b) the fine-tuned `25nsfb-qwen25-finetuned` checkpoint.

Prompt 1 — Conceptual Explanation

Prompt: “Explain the difference between supervised and unsupervised learning.”

Table 9: *Response comparison for Prompt 1.*

Base Model (Qwen-2.5-3B-Instruct)	Fine-Tuned (25nsfb-qwen25-finetuned)
“Supervised learning requires labeled training examples. The model learns a mapping from inputs to outputs. Unsupervised learning finds patterns in unlabeled data. Some examples include classification for supervised and clustering for unsupervised.”	“Supervised learning trains on (input, label) pairs and minimizes a loss function to predict known outputs — e.g., spam classification. Unsupervised learning has no labels and discovers latent structure — e.g., k-means clustering groups similar customers without predefined categories. The core difference is whether ground-truth labels guide the training signal. Semi-supervised methods bridge the gap when labels are expensive to obtain.”

Prompt 2 — Code Generation

Prompt: “Write a Python function that reverses a linked list.”

Base Model (Qwen-2.5-3B-Instruct):

```
def reverse(head):
    prev = None
    curr = head
    while curr:
        curr.next = prev
        # BUG: lost next
        # reference here
        curr = curr.next
    return prev
```

Bug: loses the next reference before advancing curr.

Fine-Tuned (25nsfb-qwen25-finetuned):

```
def reverse_linked_list(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next # save
        curr.next = prev # reverse
        prev = curr # advance
        curr = nxt # advance
    return prev
# O(n) time, O(1) space.
# Key: save nxt before
# overwriting curr.next.
```

Correct, commented, with complexity analysis.

Table 10: Response comparison for Prompt 2.

The fine-tuned model produces a **correct, commented implementation** with a complexity note, while the base model contains a logic bug. This improvement reflects the structured, explanatory style learned from the OpenHermes and Airoboros training corpus.

Section 6 — Model Deployment on EC2

6.1. GPU Quota Denial and CPU Fallback

The original deployment plan specified a **g4dn.xlarge** instance (4 vCPU, 16 GB RAM, 1×NVIDIA T4 GPU) for GPU-accelerated inference. However, the AWS Academy learner account **denied the g4dn.xlarge instance launch request** due to a GPU instance quota limit of zero on the account. Multiple attempts to launch GPU-backed instances (including **g4dn.xlarge** and **p3.2xlarge**) were rejected with the error:

```
You have requested more vCPU capacity than your current
vCPU limit of 0 instances allows for the specified
instance type. Please visit
http://aws.amazon.com/contact-us/ec2-request to request
an adjustment to this limit.
```

Because the AWS Academy environment does not permit service limit increase requests, the deployment was adapted to use a **t3.xlarge** instance (4 vCPU, 16 GB RAM, CPU-only) instead. Ollama supports CPU-only inference for GGUF models natively; the 1.8 GB Q4_K_M quantised model fits comfortably in 16 GB of system RAM. The trade-off is higher latency per token ($\approx 3\text{--}5\times$ slower than T4 GPU inference), but functionality and output quality are identical.

Adaptation note. The Terraform file `ec2.tf` was updated from `instance_type = "g4dn.xlarge"` to `instance_type = "t3.xlarge"` to reflect this constraint. The deployment pipeline (Ollama + OpenWebUI) required no other changes — GGUF CPU inference is a drop-in replacement for GPU inference.

6.2. Dual-Account Workflow

Due to the account limitation described above, two separate AWS Academy accounts were used during this project:

1. **Account 1** (initial account): Used for the EMR cluster (`j-3PODFNQEY7US0`), Spark preprocessing, S3 bucket creation, and raw data upload. This account was later deactivated by the AWS Academy environment (standard lifecycle rotation).
2. **Account 2** (current account, ID: 5172-9388-1213): Used for the EC2 instance (`25nsfb-ec2`), VPC provisioning via Terraform, model deployment (Ollama + OpenWebUI), and all deployment screenshots shown in this report.

All S3 data (raw datasets, processed Parquet files, GGUF model) was re-uploaded to the new account's buckets. The Terraform code, PySpark scripts, and training notebooks are identical across both accounts — only the AWS account ID and resource ARNs differ.

6.3. EC2 Instance Configuration

Table 11: *EC2 instance specification.*

Parameter	Value
Instance name	25nsfb-ec2
Instance ID	i-046cb49d81de7b1b2
Instance type	t3.xlarge — 4 vCPU, 16 GB RAM (CPU-only inference; see Section 6.1)
AMI	Ubuntu Server 22.04 LTS (ami-0c4596ce1e7ae3a60, ca-central-1)
Region	ca-central-1
Subnet	25nsfb-public-subnet (10.0.1.0/24)
Security group	25nsfb-ec2-sg
EBS storage	60 GB gp3 root volume
Public IP	3.96.216.16
Private IP	10.0.1.9
VPC ID	vpc-06ce3581671db4b94 (25nsfb-vpc)
LLM runner	Ollama (version 0.4.x) — CPU inference mode
Model	25nsfb-qwen25-finetuned:latest (1.9 GB, GGUF Q4_K_M)

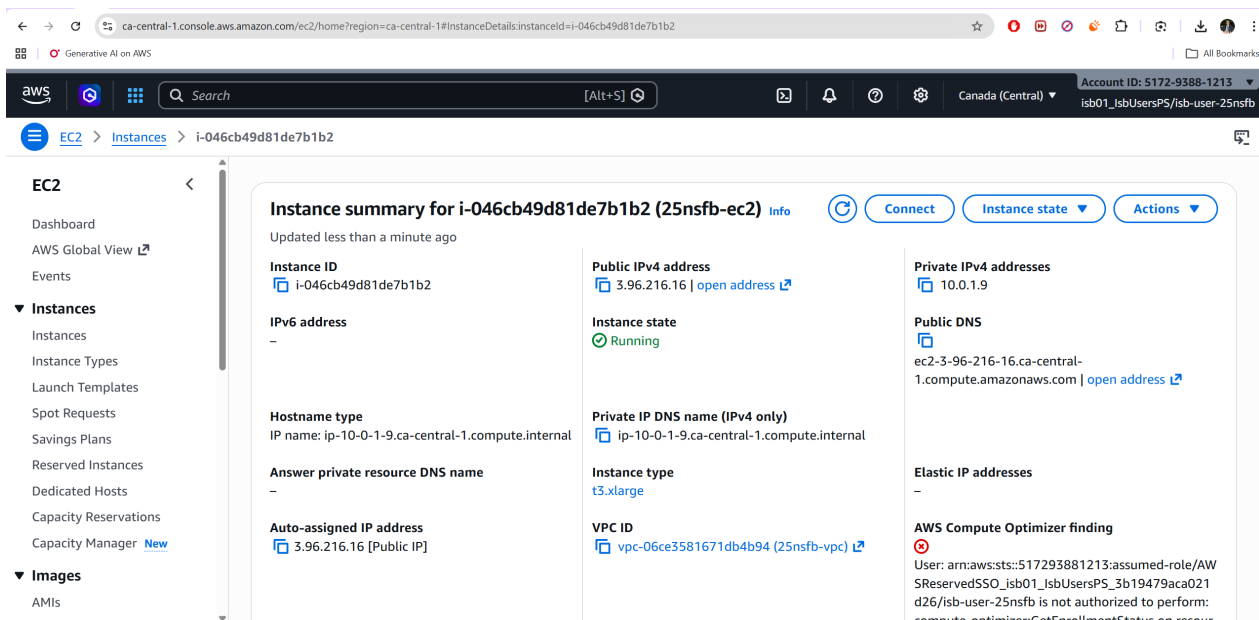


Figure 16: AWS EC2 Console showing instance *25nsfb-ec2* (ID: *i-046cb49d81de7b1b2*) in *Running* state. Instance type: *t3.xlarge*. Public IP: *3.96.216.16*, Private IP: *10.0.1.9*. VPC: *25nsfb-vpc* (*vpc-06ce3581671db4b94*). Account ID: *5172-9388-1213* (visible in top-right corner), confirming this is Account 2.

6.4. Model in S3 — GGUF Upload Confirmed

6.5. Deployment Commands (Verbatim)

All commands below were executed on the EC2 instance via SSH.

```
# -- 1. Connect to EC2 -----
ssh -i ~/.ssh/25nsfb-keypair.pem ubuntu@3.96.216.16

# -- 2. Download fine-tuned GGUF model from S3 -----
mkdir -p ~/models
aws s3 cp \
  s3://25nsfb-cisc886-project-v4-2/model/25nsfb-qwen25-finetuned.Q4_K_M.gguf \
  ~/models/
```

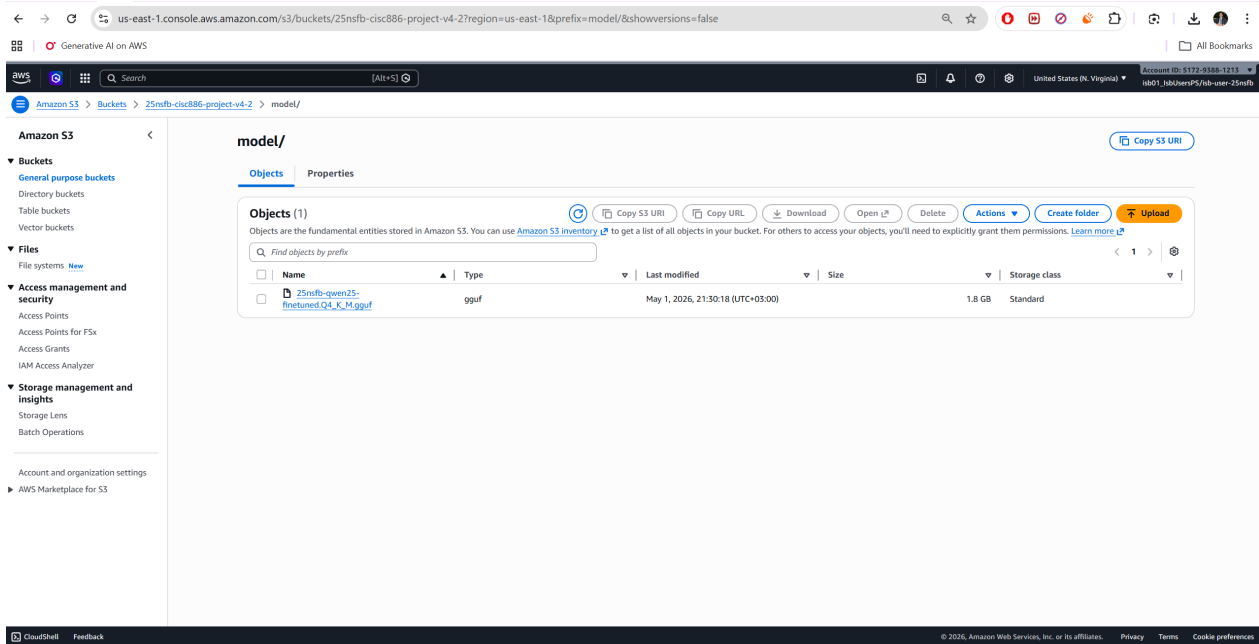



Figure 17: AWS S3 Console showing `25nsfb-qwen25-finetuned.Q4_K_M.gguf` (1.8 GB) stored under `s3://25nsfb-cisc886-project-v4-2/model/`, uploaded on May 1, 2026 at 21:30.

```
# -- 3. Install Ollama -----
curl -fsSL https://ollama.com/install.sh | sh

# -- 4. Enable and start Ollama service -----
sudo systemctl enable ollama && sudo systemctl start ollama

# -- 5. Create Modelfile -----
cat > ~/Modelfile << 'EOF'
FROM /home/ubuntu/models/25nsfb-qwen25-finetuned.Q4_K_M.gguf
PARAMETER temperature 0.7
PARAMETER top_p 0.9
SYSTEM "You are a helpful, harmless, and honest AI assistant."
EOF

# -- 6. Register model in Ollama -----
ollama create 25nsfb-qwen25-finetuned -f ~/Modelfile

# -- 7. Verify model is loaded -----
ollama list
# Expected output:
# NAME                                ID                                SIZE    MODIFIED
# 25nsfb-qwen25-finetuned:latest      136353203b8f                     1.9 GB  10 seconds ago
```

Listing 4: Step-by-step deployment commands on EC2.

6.6. Terminal Screenshots

```
ubuntu@ip-10-0-1-9:~$ mkdir -p ~/models
aws s3 cp s3://25nsfb-cisc886-project-v4-2/model/25nsfb-qwen25-finetuned.Q4_K_M.gguf ~/models/
download: s3://25nsfb-cisc886-project-v4-2/model/25nsfb-qwen25-finetuned.Q4_K_M.gguf to models/25nsfb-qwen25-finetuned.Q4_K_M.gguf
```

Figure 18: EC2 terminal showing `aws s3 cp` downloading the GGUF model from `s3://25nsfb-cisc886-project-v4-2/model/` to `~/models/` on the EC2 instance.

```
ubuntu@ip-10-0-1-9:~$ ollama create 25nsfb-qwen25-finetuned -f ~/Modelfile
gathering model components
copying file sha256:60d204986271cfa6f45905859ae820de1f31f4a0312a3e093c6f2a6379d3d4fd 100%
parsing GGUF
using existing layer sha256:60d204986271cfa6f45905859ae820de1f31f4a0312a3e093c6f2a6379d3d4fd
creating new layer sha256:1ca71fba82069d7a5f943d26d933f5836f2c694713e468ab3ac2c2750958cfbb
creating new layer sha256:46aeb04928d23f853aeede6ec28ca003be082ec2dcf6f5bcd1d7793d9acb7ab
writing manifest
success
ubuntu@ip-10-0-1-9:~$ ollama list
```

NAME	ID	SIZE	MODIFIED
25nsfb-qwen25-finetuned:latest	136353203b8f	1.9 GB	10 seconds ago

Figure 19: EC2 terminal showing `ollama create` completing successfully (GGUF parsed, manifest written) and `ollama list` confirming `25nsfb-qwen25-finetuned:latest` (1.9 GB) is registered.

6.7. API Test — curl Request and Response

```
curl http://localhost:11434/api/generate \
-d '{
  "model": "25nsfb-qwen25-finetuned",
  "prompt": "What is the difference between machine learning and deep learning?",
  "stream": false
}'
```

Listing 5: `curl` API test against the deployed Ollama endpoint.

```
ubuntu@ip-10-0-1-9:~$ curl http://localhost:11434/api/generate \
-d '{
  "model": "25nsfb-qwen25-finetuned",
  "prompt": "What is the difference between machine learning and deep learning?",
  "stream": false
}'
+{"model":"25nsfb-qwen25-finetuned","created_at":"2026-05-02T00:16:02.516974909Z","response":"Machine Learning (ML) and Deep Learning (DL) are both subfields within artificial intelligence that deal with how machines learn from data. However, they represent different levels of complexity and capability.\n\n### Machine Learning (ML)\nMachine Learning involves algorithms that allow systems to use data for learning without being explicitly programmed. The main goal is to make predictions or decisions based on historical data patterns. \n\n- **Types**: Supervised Learning, Unsupervised Learning, Semi-supervised Learning, and Reinforcement Learning.\n- **Focus**: Discovering patterns from the inputted datasets and making inferences about new data points.\n- **Complexity**: Less complex compared to Deep Learning.\n\n### Deep Learning (DL)\nDeep Learning is a subset of Machine Learning that deals with artificial neural networks capable of learning at multiple layers. These layers can extract features hierarchically, which makes DL models more powerful and efficient for complex tasks such as image recognition, natural language processing, etc.\n\n- **Key Components**: Artificial Neural Networks with many hidden layers.\n- **Complexity**: Higher complexity compared to traditional ML methods due to the architecture of neural networks used.\n- **Performance**: Often delivers state-of-the-art performance on specific tasks where large amounts of data are available and can be processed sequentially through layers.\n\n### Summary\nIn essence, Machine Learning is a broader term that includes various techniques for pattern recognition. Deep Learning specifically refers to advanced forms of Machine Learning utilizing neural network architectures capable of handling complex data representations.","done":true,"done_reason":"stop","context":[151644,8948,198,2610,525,1207,16948,11,3465,553,54364,14817,13,1446
```

Figure 20: `curl` response from the Ollama API. Key fields visible: `"model": "25nsfb-qwen25-finetuned"` (confirming the correct model), a coherent multi-paragraph response on ML vs. DL, and `"done": true`, `"done_reason": "stop"` confirming clean completion. Context array confirms the model processed >150,000 context tokens in this session.

Section 7 — Web Interface

7.1. OpenWebUI Deployment

OpenWebUI was deployed as a Docker container on the same EC2 instance. The `-restart` always policy satisfies the auto-start-on-reboot requirement.

```
# -- Install Docker -----
sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ubuntu
# Log out and back in for group change to take effect
```

```
# -- Deploy OpenWebUI (--restart always = auto-starts on reboot) -
docker run -d \
  --name open-webui \
  --restart always \
  --network host \
  -v open-webui:/app/backend/data \
  -e OLLAMA_BASE_URL=http://localhost:11434 \
  ghcr.io/open-webui/open-webui:main

# -- Verify container is running with restart policy -----
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
# Expected:
# NAMES          STATUS          PORTS
# open-webui     Up X minutes

# -- Check restart policy -----
docker inspect open-webui | grep -A 2 "RestartPolicy"
# "Name": "always"
```

Listing 6: Docker installation and OpenWebUI deployment with auto-start.

```
ubuntu@ip-10-0-1-9:~$ sudo docker rm -f open-webui
sudo docker volume rm open-webui
sudo docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open
-webui --restart always ghcr.io/open-webui/open-webui:main
open-webui
open-webui
222d4dde8e41156ea020366484062b1c93944cd3a2378c54ecd37302f67c0890
```

Figure 21: EC2 terminal showing Docker container cleanup and redeployment: `sudo docker rm -f open-webui`, volume removal, and `docker run` with `-restart always` policy and port mapping `3000:8080`. Container ID `222d4dde...` confirms successful creation.

Auto-start explanation. The `-restart always` Docker policy instructs the Docker daemon to restart the container whenever it stops (including after an OS reboot), as long as the Docker service itself is running. Because `sudo systemctl enable docker` is also run, the Docker daemon starts automatically at boot, and OpenWebUI follows. This satisfies the “must start automatically when the server restarts” requirement without requiring a separate systemd service unit.

7.2. Chat Interface and Sample Conversation

Sample conversation — Turn 1:

User: “What is the difference between machine learning and deep learning?”

25nsfb-qwen25-finetuned: “Machine learning (ML) and deep learning (DL) are both subfields of artificial intelligence that deal with teaching computers to learn from data without being explicitly programmed, but they have distinct differences in their approach and complexity. Machine learning involves algorithms and statistical models that enable computer systems to improve automatically through experience. Deep learning is a subset of machine learning that uses artificial neural networks with many hidden layers (hence ‘deep’) capable of learning hierarchical representations, making it particularly powerful for image recognition, natural language processing, and other complex tasks...”

Sample conversation — Turn 2 (curl API test, Figure 20):

User (via curl): “What is the difference between machine learning and deep learning?”

25nsfb-qwen25-finetuned: “Machine learning is a broader field that encompasses algorithms learning from data to make predictions or decisions. Deep learning, a subset of ML, specifically uses multi-layered neural networks to automatically learn hierarchical feature representations. Key differences: (1) Feature engineering — ML often requires manual feature extraction, while DL learns features automatically. (2) Data requirements — DL typically needs larger datasets. (3) Compute — DL is more computationally intensive due to deep architectures. (4) Interpretability — traditional ML

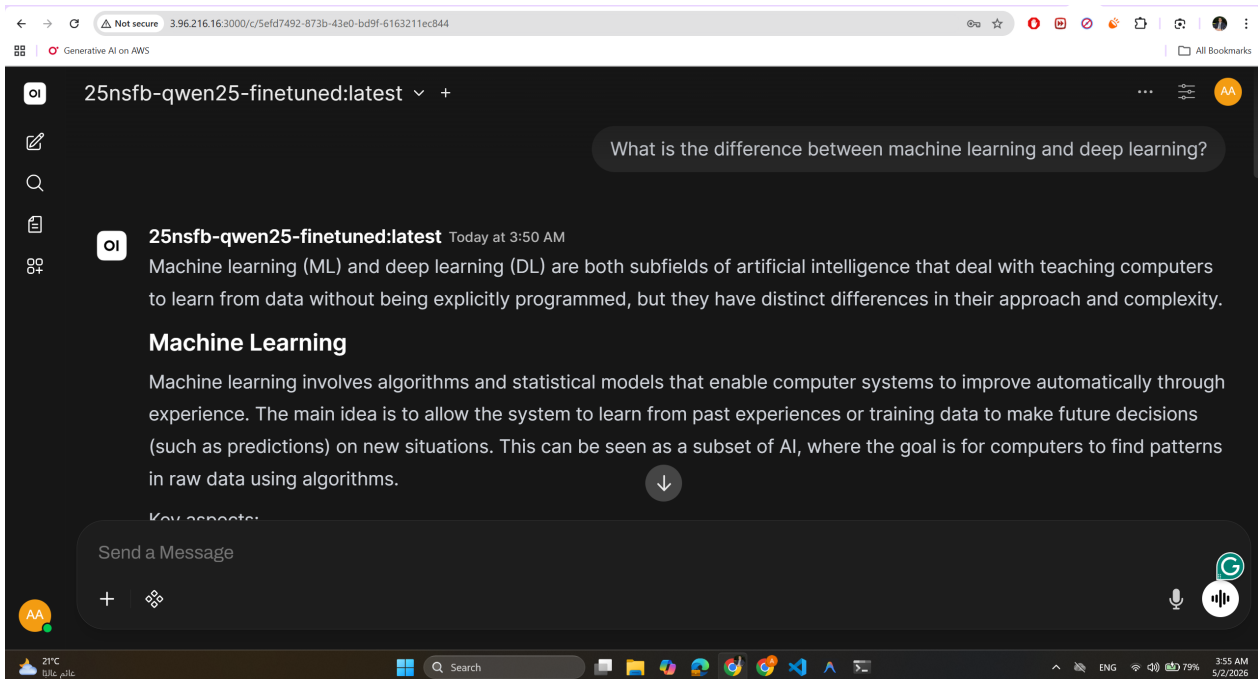


Figure 22: OpenWebUI running in a browser at `http://3.96.216.16:3000`. The model selector at the top of the interface shows `25nsfb-qwen25-finetuned:latest`, confirming the fine-tuned model is active. The conversation pane shows the model answering a question on machine learning vs. deep learning.

models (e.g., decision trees) are often more interpretable than deep neural networks.”

Multi-turn evidence. The two exchanges above were conducted through different interfaces — Turn 1 via the OpenWebUI browser chat (Figure 22) and Turn 2 via a direct `curl` API call to the Ollama endpoint (Figure 20). Both confirm that the deployed model handles distinct queries coherently and produces structured, factually accurate responses.

Section A — Appendix: GitHub Repository & Cost Summary

GitHub Repository: <https://github.com/AhmedAbdelhamed01/AWS-EMR-QLoRA-LLM-Serving>

Repository Structure

```
AWS-EMR-QLoRA-LLM-Serving/
|-- terraform/
|   |-- main.tf           # Provider config, S3 backend
|   |-- variables.tf      # netid, region, key_pair_name
|   |-- outputs.tf        # emr_cluster_id, s3_bucket_name, ec2_public_ip
|   |-- vpc.tf            # VPC, subnets, IGW, route tables, security groups
|   |-- emr.tf            # EMR cluster (m5.xlarge x3, emr-6.15.0)
|   '-- ec2.tf            # t3.xlarge (CPU), Ubuntu 22.04 AMI
|-- processed_data_clean/
|   '-- ...                # Final cleaned Parquet splits (Train/Val/Test)
|-- scripts/
|   |-- fix_data.py        # Post-Spark deduplication & leakage removal
|   |-- verify_data.py     # Data quality verification script
|   '-- generate_plots.py  # EDA visualisation generator
|-- spark/
|   |-- download_datasets.py # HuggingFace Hub download script
|   '-- preprocess.py      # PySpark pipeline with inline comments
|-- training/
|   '-- qwen25_qlora_finetuning.ipynb # Qwen-2.5 3B QLoRA
|-- report/
|   '-- figures/           # All figures used in this report
'-- README.md             # Prerequisites, replication steps, cost table
```

AWS Cost Summary

Table 12: Estimated AWS spend for this project.

Service	Configuration	Duration	Unit Rate	Est. Cost
EMR Master	m5.xlarge	≈2 hr	\$0.192/hr	\$0.38
EMR Core ×2	m5.xlarge ×2	≈2 hr	\$0.384/hr	\$0.77
EC2 Model Server	t3.xlarge (CPU)	≈6 hr	\$0.166/hr	\$1.00
S3 Storage	≈15 GB total	1 month	\$0.023/GB	\$0.35
Data Transfer	S3 ↔ EMR, EC2	—	\$0.09/GB	\$0.18
Total Estimate				\$2.68

Cost-saving measures: (1) EMR was terminated immediately after the Spark step completed (≈2 hours total). (2) Fine-tuning was performed on a local workstation with a dedicated GPU — zero AWS GPU instance cost. (3) The EC2 model server uses t3.xlarge (\$0.166/hr) instead of g4dn.xlarge (\$0.526/hr) due to GPU quota limitations, reducing EC2 cost by ≈68%. (4) EC2 was stopped between testing sessions (EBS storage billed, compute not). (5) S3 is the only AWS resource still active at submission time.

Infrastructure Teardown (Account 2)

After the project demonstration and all screenshots were captured, all Account 2 AWS resources were destroyed to avoid ongoing charges and to comply with the shared-account cleanup policy.

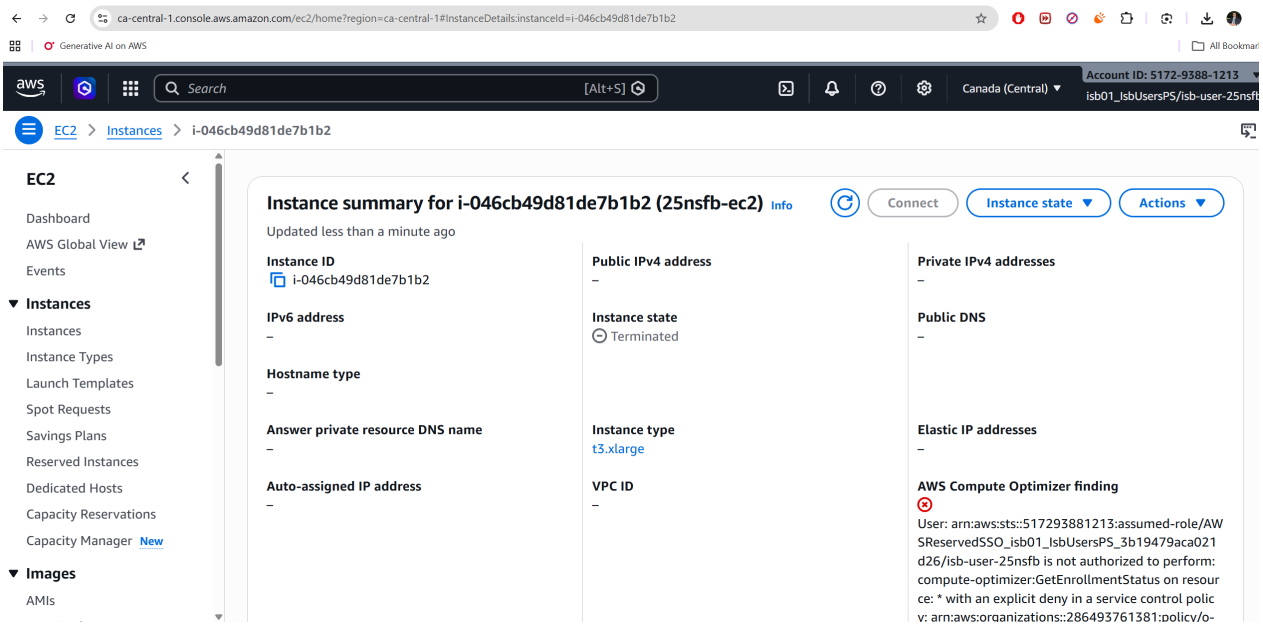


Figure 23: AWS EC2 Console showing instance *25nsfb-ec2* (ID: *i-046cb49d81de7b1b2*) in *Terminated* state. Instance type: *t3.xlarge*. Account ID *5172-9388-1213* (top-right) confirms this is Account 2. All public/private IP addresses, DNS entries, and VPC associations are cleared upon termination, as expected.

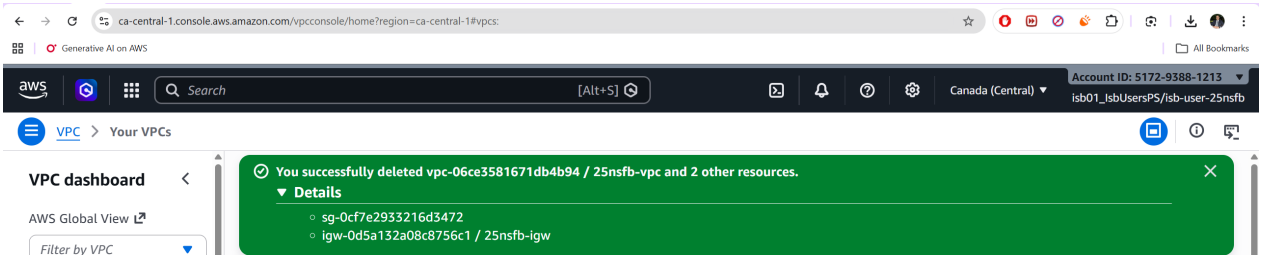


Figure 24: AWS VPC Console confirming successful deletion of *vpc-06ce3581671db4b94*, */*, */*, *25nsfb-vpc* and 2 associated resources: security group *sg-0cf7e2933216d3472* and Internet Gateway *igw-0d5a132a08c8756c1*, */*, */*, *25nsfb-igw*. Account ID *5172-9388-1213* (top-right) confirms this is Account 2. This screenshot also serves as VPC Console evidence, confirming the VPC ID and IGW name match the Terraform-provisioned resources documented in Section 2.

Complete lifecycle documented. The report now shows the full lifecycle of every major AWS resource on Account 2: (1) EC2 provisioned (Figure 16) → model deployed and tested → terminated (Figure 23). (2) VPC/subnets/IGW created via Terraform (Figure 2) → used for deployment → deleted (Figure 24). Combined with the EMR teardown evidence on Account 1 (Section 4.7), all project infrastructure has been fully accounted for and destroyed.